Technology Selection

Project: AI-Assisted Travel Planning Web Application

1. Purpose and Architectural Alignment

This review identifies suitable technologies for the components in the TripMind UML Component Diagram and compares three Python libraries for interacting with LLM services. The recommendations describe the proposed implementation, rather than completed functionality. As of 16 October 2026, React/Vite and Flask are connected. The backend validates inputs and returns a deterministic two-day London sample with structured dates and group costs. Live LLM generation and LLM-output validation are not implemented. SQLite and the data access layer remain proposed, not installed or implemented components.

2. Technologies for Each UML Component

Web Front-end

React, Vite, JavaScript/JSX, CSS and the Fetch API. Retain the existing interface for destination, dates, budget, travellers and interests. React displays daily itineraries and manages loading and error states; Fetch sends JSON requests to the backend.

API Layer (Flask Routes)

Python and Flask are implemented. POST /api/trips/generate accepts preferences and returns structured sample JSON; GET /api/health reports sample mode. Input validation runs inside generation. Separate validation and attraction endpoints are not implemented and are not required for this prototype.

Travel Business Logic

Current implementation: Python standard-library date handling and explicit input checks in validation.py, with sample generation and cost calculation in planner.py. Two consecutive dates, London, GBP, 1-4 travellers and a finite budget are checked. Over-budget samples return a flag; route feasibility and activity overlaps are not checked. Pydantic [1], decimal and attraction retrieval remain candidates, not current dependencies.

LLM Interaction Component

Planned only: a selected LLM library plus an optional schema library such as Pydantic. Build prompts from preferences and constraints, call the external service, parse responses and handle failures. Define a shared itinerary schema for daily activities, estimated costs and uncertainty notes. Keep API credentials in backend environment configuration.

Data Access Layer

Proposed only: Python sqlite3. Encapsulate queries for attractions, cities and categories. The built-in library is sufficient for a small dataset. SQLAlchemy is an alternative if more complex database relationships justify an object-relational mapping layer.

Database (SQLite)

Proposed only: SQLite. Store a small, curated attraction dataset with source links and verification dates. User history remains optional. Existing browser localStorage saving is separate from this proposed server-side database.

External LLM Services

Unselected candidates: OpenAI, Anthropic or Google. These are model providers, whereas the Python libraries are integration tools. Select one service for the initial application; the diagram does not require all providers to be connected.

3. Role and Limits of Pydantic

Pydantic is a proposed supporting library, not a current dependency or a separate UML component. It can validate both user requests and model outputs. Correct structure does not establish factual accuracy: prices, opening hours and travel feasibility still require evidence or explicit uncertainty labels.

4. Comparison of Python LLM Libraries

OpenAI Python SDK (openai)

- Strengths: Direct access to OpenAI services and structured-output helpers that parse responses into Pydantic models [2]. A small integration is straightforward to inspect and explain.

- Limitations: Primarily provider-specific. Other services may require separate integrations, and structured-output availability depends on the chosen model.

- Suitability: The preferred starting point if OpenAI is selected for the MVP.

LiteLLM (litellm)

- Strengths: A common Python interface across multiple providers, with normalized responses and errors. Useful for changing providers while retaining similar application code [3].

- Limitations: Adds an abstraction layer. Provider capabilities and supported schema options still differ; compatibility must be checked for each model [4].

- Suitability: A strong candidate when cross-provider experiments are central to the dissertation.

LangChain (langchain and provider integration packages)

- Strengths: Standard model interfaces, structured-output support and facilities for composing model workflows [5].

- Limitations: More concepts and dependencies to manage. Its broader workflow features offer limited benefit for a simple request, generation and validation pipeline.

- Suitability: Consider if requirements develop into more complex workflows; not necessary for the initial MVP.

5. Preliminary Recommendation

Retain the connected React/Vite and Flask implementation. Consider Pydantic for future LLM-output schemas. sqlite3/SQLite remain an optional storage proposal; the current sample loads backend/sample_itinerary.json and saved plans use browser localStorage. Begin with a direct provider SDK once the provider is chosen; use LiteLLM instead if provider comparison is a core requirement. Isolate model calls in one backend module so the integration can be replaced. Keep weather, maps, booking, accounts and complex multi-agent workflows outside the initial scope. Optional routes and user history should only be implemented if needed. Update the UML if the final storage or component responsibilities change.

6. Proposed Evaluation

This comparison is based on documentation and project fit; no library benchmark has been run. Where supported, compare libraries using the same model, prompt, schema and travel inputs, with equivalent settings and repeated trials.

- Record setup effort, structured-output success, error handling, latency, token usage and ease of switching providers. Record versions and retry settings.

- Evaluate itinerary relevance, budget consistency and date coverage separately from integration-library suitability. Model and prompt differences can affect these outcomes. Current API tests use unittest and Flask's test client [6]; five test methods passed on 16 October 2026. pytest is only an optional future alternative. Retain prompts, model identifiers, raw outputs and evaluation results in JSONL or CSV for reproducible project evidence.

7. References

[1] Pydantic documentation. https://docs.pydantic.dev/latest/

[2] OpenAI: Structured model outputs. https://developers.openai.com/api/docs/guides/structured-outputs

[3] LiteLLM: Python SDK. https://docs.litellm.ai/docs/python_sdk

[4] LiteLLM: Structured outputs. https://docs.litellm.ai/docs/completion/json_mode

[5] LangChain: Models. https://docs.langchain.com/oss/python/langchain/models

[6] Flask: Testing Flask applications. https://flask.palletsprojects.com/en/stable/testing/

Original documentation review: 14 October 2026 (references retained, not re-reviewed in this audit). Implementation status checked: 16 October 2026. Architecture reference: updated TripMind UML Component Diagram; planned components are explicitly marked.